

The background of the slide is a photograph of a historic building, likely the Friedrich-Schiller-Universität Jena, featuring a prominent clock tower with a blue roof and white clock faces. The building is surrounded by lush green trees, and the scene is captured in bright daylight. A semi-transparent white box is overlaid on the left side of the image, containing the title and authors.

Parallel Computing OpenMP Task

T. Bosse, R. Domogalla, H. M. Bucker
Advanced Computing, FSU Jena

OpenMP Tasks

Tasks allow to define sections of code that can be executed in parallel. Each task is a unit of work that the runtime system will schedule to run on available threads.

Task Creation:

- Defined using the `#pragma omp task` directive.
- Tasks can be created dynamically and can execute concurrently.

Task Execution:

- Tasks are scheduled and executed by the runtime system, possibly on different threads.
- The order of task execution is not guaranteed.

Synchronization:

- `#pragma omp taskwait` is used to wait for the completion of all child tasks.
- Ensures that all spawned tasks in the program are completed at a given point.

```
#pragma omp parallel
{
    #pragma omp single
    {
        #pragma omp task
        {
            // Task 1: perform some computation
        }

        #pragma omp task
        {
            // Task 2: perform some other computation
        }
    }
    // Implicit barrier ensures all child tasks to be
    completed
}
```

Task Pragmas

```
#pragma omp task [clause list]  
/*Structure Block*/
```

Clause Type	Explanation
if(cond)	Specify that the task exists if the condition is true
mergeable	Specify that the task is mergeable with other mergeable tasks
final(cond)	Specify that the task is evaluated last if the condition is true
depend(dep-type: list)	Specify variable dependency that need to be evaluated before this task
priority(value)	Specify priority of a task to be evaluated

Traverse Example

Recursive traversal of a binary tree using OpenMP Tasks.

- Traversal of left and right subtree is done in parallel, i.e. considered as two tasks that will split into several tasks themselves
- Each task has its own copy to the pointer of the current node

```
// Example tasking.1.c from OpenMP Examples  
// Version 5.2  
// OpenMP Architecture Review Board  
  
struct node {  
    struct node *left;  
    struct node *right;  
};  
  
extern void process(struct node *);  
void traverse( struct node *p )  
{  
    if (p->left)  
        #pragma omp task // p is firstprivate by default  
        traverse(p->left);  
    if (p->right)  
        #pragma omp task // p is firstprivate by default  
        traverse(p->right);  
    process(p);  
}
```

Task Priority

- Single region generates all of the explicit tasks
- Tasks are executed by threads within the current thread team
- Higher priority means earlier execution of task

```
// Example task_priority.1.c from OpenMP Examples
// Version 5.2
// OpenMP Architecture Review Board

void compute_array (float *node, int M);

void compute_matrix (float *array, int N, int M)
{
    int i;
    #pragma omp parallel private(i)
    #pragma omp single
    {
        for (i=0; i<N; i++) {
            #pragma omp task priority(i)
            compute_array(&array[i*M], M);
        }
    }
}
```

Scoping for Tasks

```
#pragma omp task [clause list]
/*Structure Block*/
```

Clause Type	Example	Explanation
private(list)	<code>#pragma omp task private(a,b)</code>	Specify the variables in the list to be private
firstprivate(list)	<code>#pragma omp task firstprivate(a,b)</code>	Specify the variables in the list to be firstprivate
shared(list)	<code>#pragma omp task shared(a,b)</code>	Specify the variables in the list to be shared
default(none)	<code>#pragma omp task default(none)</code> <code>private(a) shared(b)</code>	Specify the data scope of all variables - no default
default(shared)	<code>#pragma omp task default(shared)</code>	Specify that by default all variables are shared
...		...

Task Scoping Example

```
#include <stdio.h>
#include <omp.h>

static int a=1;
void main() {
    int b=3;
    int c=3;
    #pragma omp parallel private(c)
    {
        int d = 4;
        #pragma omp task
        {
            int e = 5;
        }
    }
}
```

Explain the scoping of the variables a, b, c, d, and e!

Hint: By default, scoping of the variables for task is similar to threads, namely,

- local variables are private
- Static and global variables are shared

But:

- task variables are first-private unless shared
- (Only) Shared attribute is inherited
- Orphan task variables are first-private.

Fibonacci Example

Recursive computation of the n-th Fibonacci number using OpenMP tasks.

- n-th Fibonacci number is a sum of two predecessors i and j
- i and j are computed in parallel by recursively spawning two different tasks.
- Both tasks share the same memory for i and j.
- The omp taskwait guarantees that the spawning task waits until both spawned tasks for i and j are finished before returning to it.

```
// Example tasking.4.c from OpenMP Examples  
// Version 5.2  
// OpenMP Architecture Review Board
```

```
int fib(int n) {  
    int i, j;  
    if (n<2)  
        return n;  
    else {  
        #pragma omp task shared(i)  
        i=fib(n-1);  
        #pragma omp task shared(j)  
        j=fib(n-2);  
        #pragma omp taskwait  
        return i+j;  
    }  
}
```


Task Scoping Example

```
#include <stdio.h>
#include <omp.h>

static int a=1;
void main() {
    int b=3;
    int c=3;
    #pragma omp parallel private(c)
    {
        int d = 4;
        #pragma omp task
        {
            int e = 5;
        }
    }
}
```

Lookup and explain the scoping of the variables a, b, c, d, and e!

Hint: By default, scoping of the variables for tasks is similar to scoping for threads, namely,

- Local variables are private.
- Static and global variables are shared.

But:

- Task variables are first-private unless shared.
- (Only) Shared attribute is inherited.
- Orphan task variables are first-private.

Task Dependency Example

//Example originating from a question on stack overflow

```
#include <stdio.h>
#include <omp.h>
#include <unistd.h>
int main() {
    int x,y;
    #pragma omp parallel num_threads(8)
    {
        #pragma omp single nowait
        {
            #pragma omp task depend (out:x) //task1
            {
                x=1;
            }
        }
        #pragma omp task depend(in:x) depend(out:y) //task2
        {
            sleep(2);
            y=x+1;
        }
    }
}
```

```
#pragma omp task depend (inout:x) //task3
{
    x++;
    printf("task3(x): %d\n" , x);
}
#pragma omp task depend (in:x,y) //task4
{
    printf("task4 (x+y): %d\n" , x+y);
}
}
return 0;
}
```

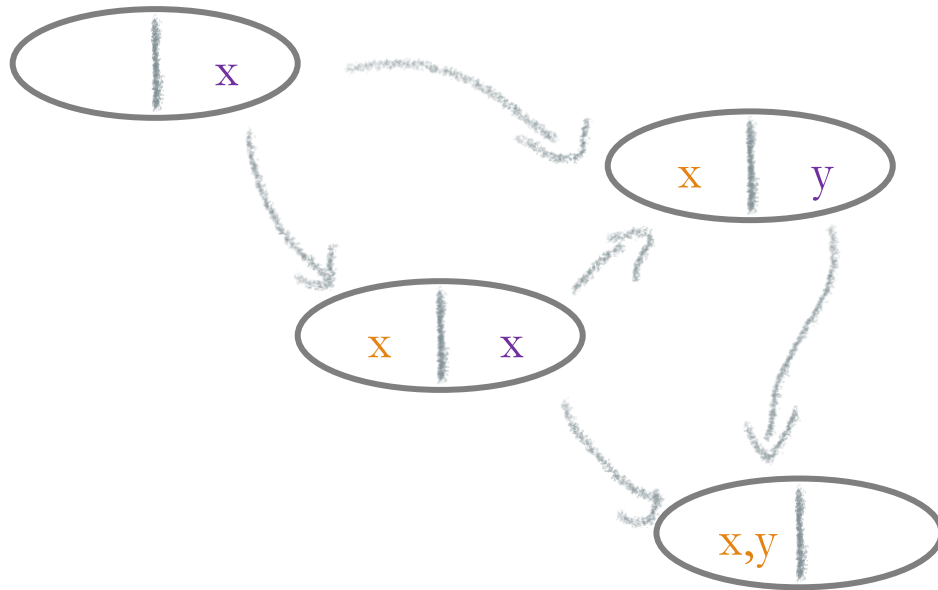
➤ `gcc -o OpenMP-TaskDepend -fopenmp OpenMP-TaskDepend.c; ./OpenMP-TaskDepend`

```
*****
😊😊😊😊 FIND OUT AND EXPLAIN THE RESULT 😊😊😊😊
*****
```

Task Dependency Example

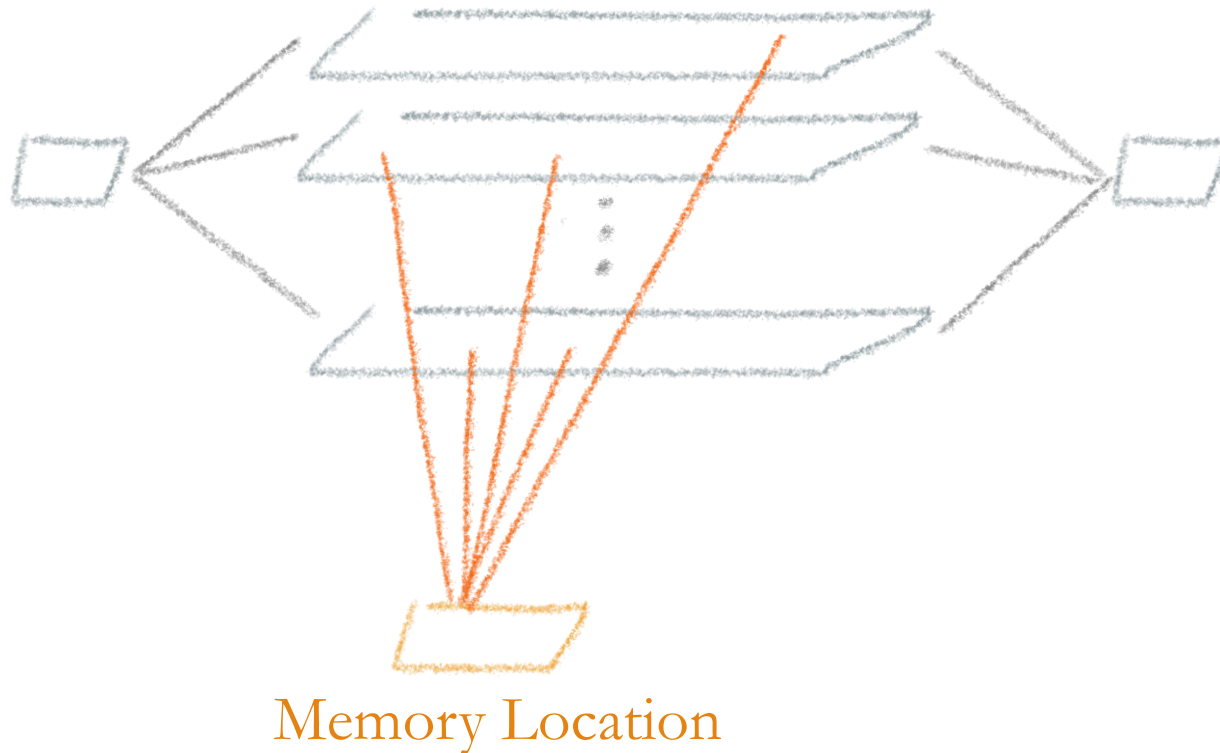


For each task, one can specify **On** and **Out** dependencies



```
#pragma omp parallel num_threads(8)
{
  #pragma omp single nowait
  {
    #pragma omp task depend (out:x) //task1
    {
      x=1;
    }
    #pragma omp task depend(in:x) depend(out:y) //task2
    {
      sleep(2);
      y=x+1;
    }
    #pragma omp task depend (inout:x) //task3
    {
      x++;
      printf("task3(x): %d\n" , x);
    }
    #pragma omp task depend (in:x,y) //task4
    {
      printf("task4 (x+y): %d\n" , x+y);
    }
  }
}
```

Race Conditions



- Two or more tasks/threads access the same memory
- One of these tasks/threads writes to a memory location
- If data access is not protected by data sharing attributes or synchronization clauses it might result in unexpected behaviour
- Data race conditions hard to detect in a sandbox/debugger environment

Bernstein Condition

Let T_1 and T_2 be two tasks of a computer code. Denote by I_1 and I_2 the set of input variables that are read by task T_1 and T_2 , respectively. Analogously, denote by O_1 and O_2 the set of output variables that are modified by task T_1 and T_2 , respectively.

The two tasks T_1 and T_2 satisfy the **Bernstein Condition** if and only if

$$I_1 \cap O_2 = \emptyset$$

$$I_2 \cap O_1 = \emptyset$$

$$O_1 \cap O_2 = \emptyset$$

The Bernstein condition is crucial to avoid race conditions and helps to find parallel structures in the code !!!

Bernstein Condition Example

Task T_i	Statement	In-Variables I_i	Out-Variables O_i
T_1	$c = d * e$	$\{e, d\}$	$\{c\}$
T_2	$m = g + c$	$\{g, c\}$	$\{m\}$
T_3	$a = b + c$	$\{b, c\}$	$\{a\}$
T_4	$c = l + m$	$\{l, m\}$	$\{c\}$
T_5	$f = g \% e$	$\{g, e\}$	$\{f\}$

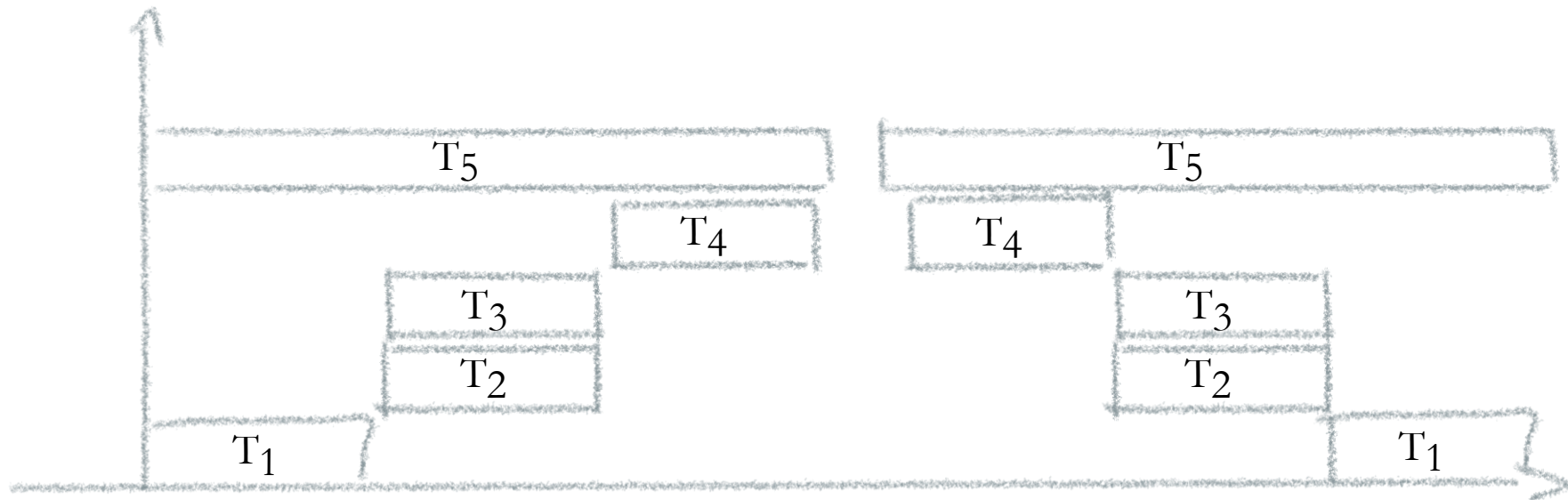
Bernstein Condition Example

Task T_i	Statement	In-Variables I_i	Out-Variables O_i
T_1	$c=d*e$	$\{e,d\}$	$\{c\}$
T_2	$m=g+c$	$\{g,c\}$	$\{m\}$
T_3	$a=b+c$	$\{b,c\}$	$\{a\}$
T_4	$c=l+m$	$\{l,m\}$	$\{c\}$
T_5	$f=g^0/e$	$\{g,e\}$	$\{f\}$

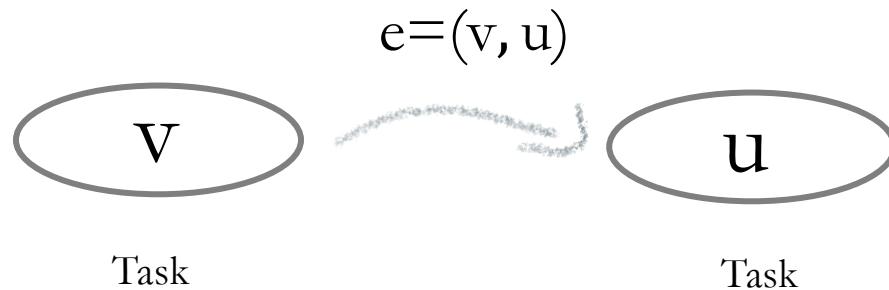
$T_i \setminus T_j$	T_1	T_2	T_3	T_4	T_5
T_1	$\{C3\}$	$\{C1\}$	$\{C1\}$	$\{C3\}$	$\{\emptyset\}$
T_2	$\{C1\}$	$\{C3\}$	$\{\emptyset\}$	$\{C1,C2\}$	$\{\emptyset\}$
T_3	$\{C1\}$	$\{\emptyset\}$	$\{C3\}$	$\{C1\}$	$\{\emptyset\}$
T_4	$\{C3\}$	$\{C1,C2\}$	$\{C2\}$	$\{C3\}$	$\{\emptyset\}$
T_5	$\{\emptyset\}$	$\{\emptyset\}$	$\{\emptyset\}$	$\{\emptyset\}$	$\{C3\}$

Bernstein Condition Example

$T_i \setminus T_j$	T_1	T_2	T_3	T_4	T_5
T_1	{C3}	{C1}	{C1}	{C3}	{ $\emptyset$ }
T_2	{C1}	{C3}	{ $\emptyset$ }	{C1,C2}	{ $\emptyset$ }
T_3	{C1}	{ $\emptyset$ }	{C3}	{C1}	{ $\emptyset$ }
T_4	{C3}	{C1,C2}	{C2}	{C3}	{ $\emptyset$ }
T_5	{ $\emptyset$ }	{ $\emptyset$ }	{ $\emptyset$ }	{ $\emptyset$ }	{C3}



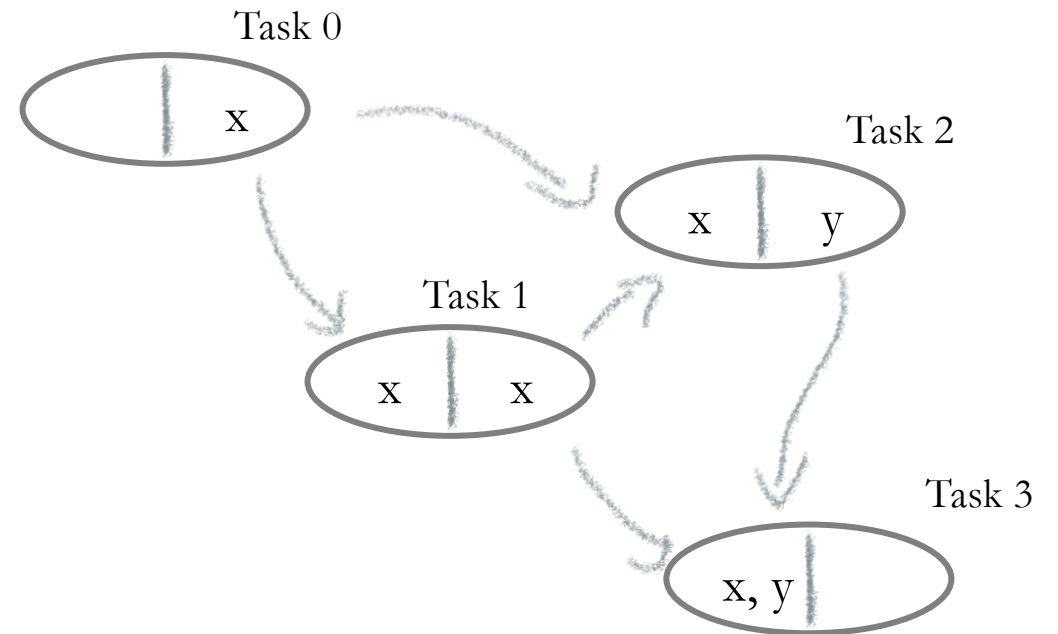
Task Dependency Graph



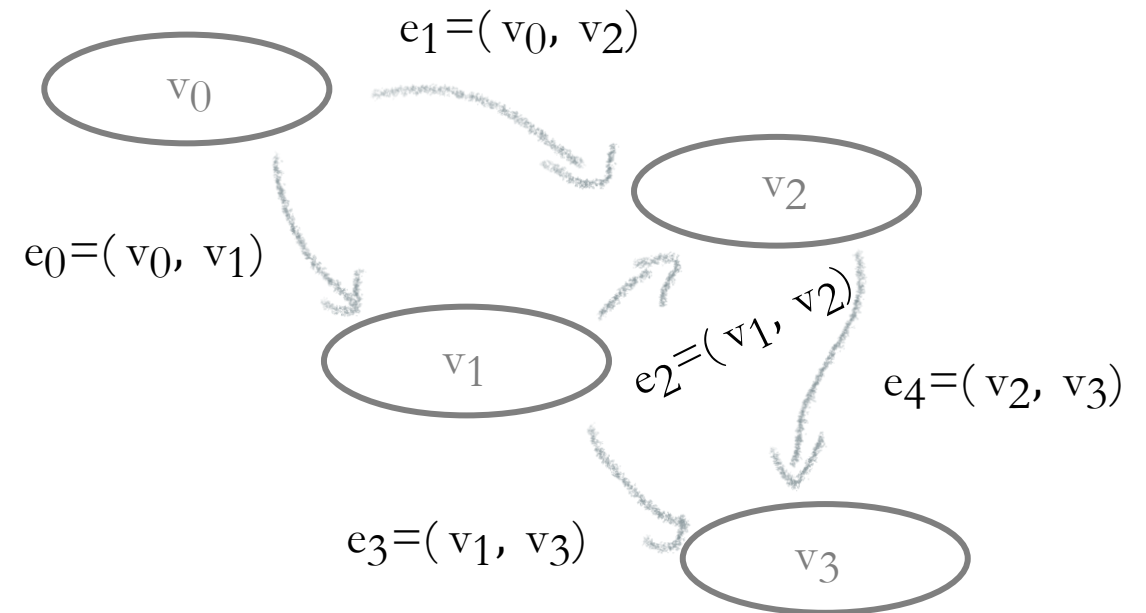
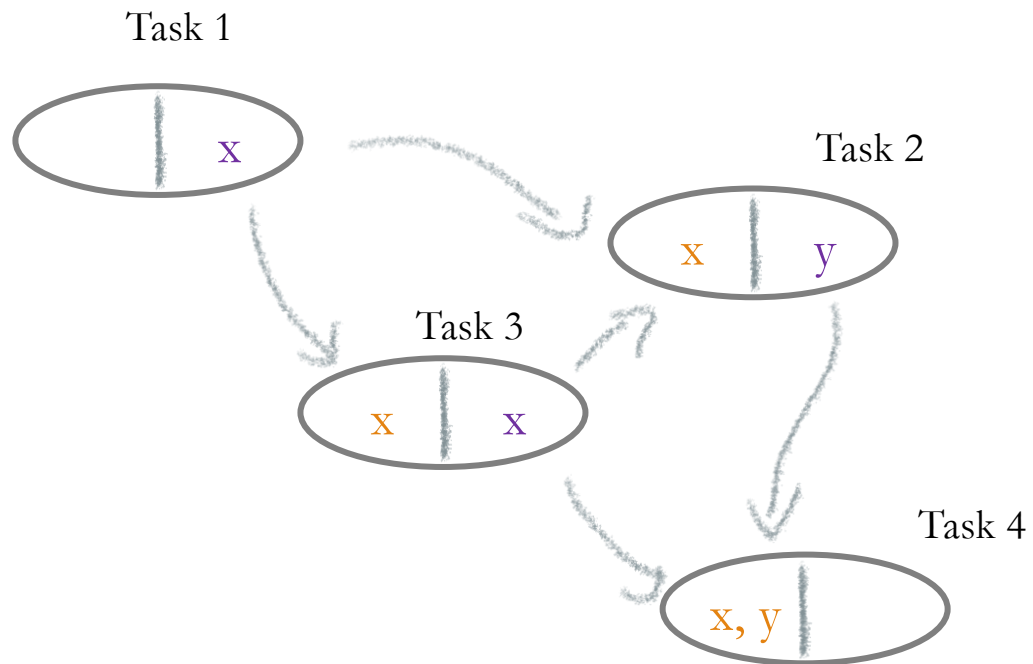
- Code can be decomposed into different smaller/atomic tasks.
- Each task might have in-/outputs.
- In-/outputs of different tasks might depend on each other.
- Dependencies/Bernstein condition induces a dependency relation.
- Induced dependence relation and tasks can be represented by a directed graph $G = (V, E)$.
- Each vertex v from the set of vertices V represents one of the tasks.
- An edge $e = (v, u)$ from vertex v to vertex u indicates that (input of) the task represented by vertex u depends on (the output of) the task represented by vertex v .

Task Dependency Graph Example

```
// OpenMP 5.2 Style - deprecated!!!
#pragma omp parallel num_threads(8)
{
  #pragma omp single nowait
  {
    #pragma omp task depend (out:x) //task1
    {
      x=1;
    }
    #pragma omp task depend(in:x) depend(out:y) //task2
    {
      sleep(2);
      y=x+1;
    }
    #pragma omp task depend (inout:x) //task3
    {
      x++;
      printf("task3(x): %d\n" , x);
    }
    #pragma omp task depend (in:x,y) //task4
    {
      printf("task4 (x+y): %d\n" , x+y);
    }
  }
}
```



Task Dependency Graph Example



$$G=(V,E)$$

$$V=\{v_0, v_1, v_2, v_3\}$$

$$E=\{e_0, e_1, e_2, e_3, e_4\}$$

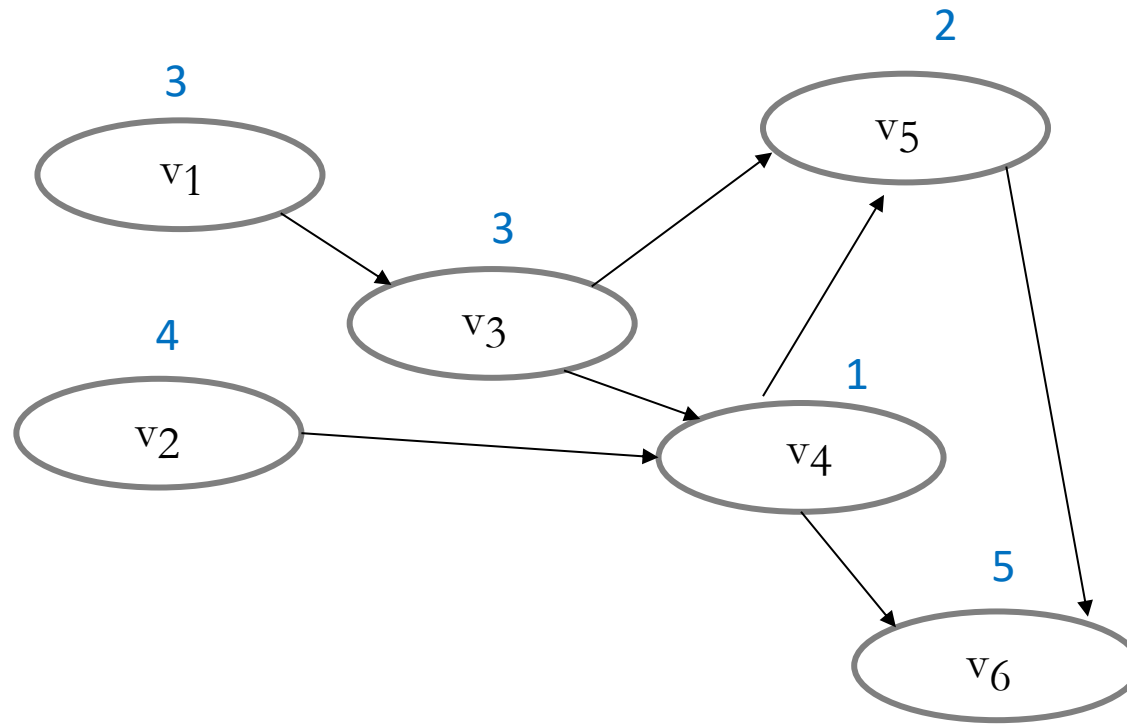
Task Dependency Graph

- Tasks have certain costs represented as weights in the graph, e.g. time for a task can be given vertices weights

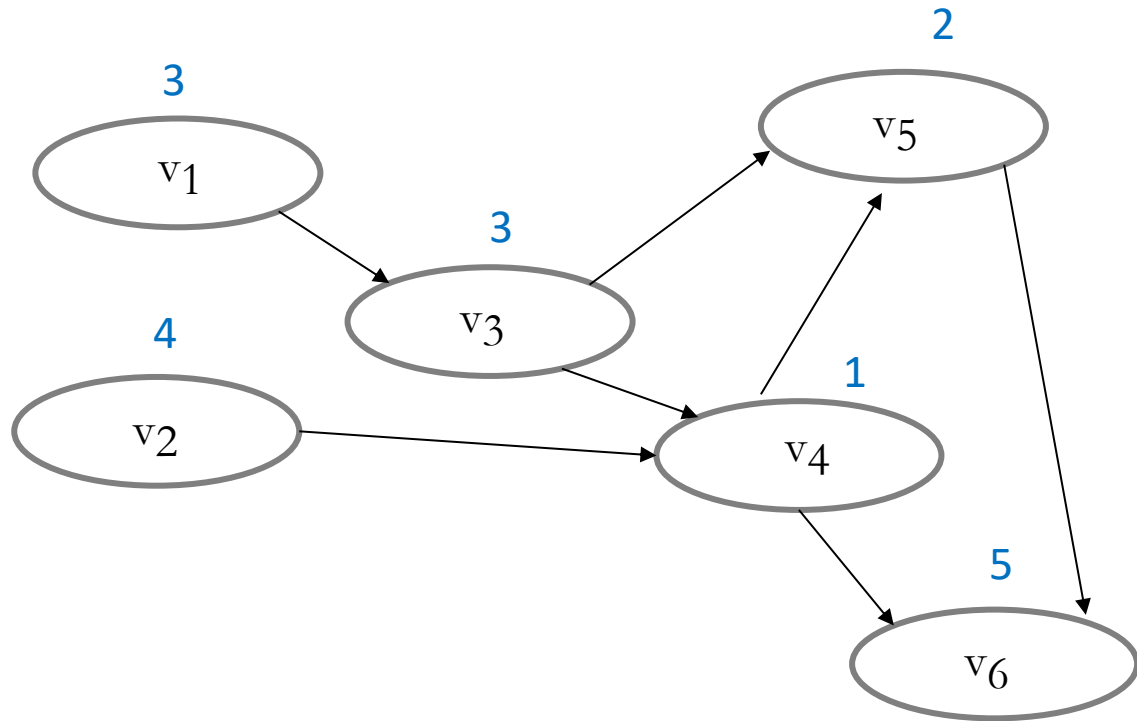
$$w: V \rightarrow \mathbb{R}$$

- Path P is a sequence of vertices, i.e. $P = (v_{i_1}, v_{i_2}, \dots, v_{i_l})$ with $(v_{i_j}, v_{i_{j+1}}) \in E$ for all $j = 1, \dots, l - 1$
- Here, the path length is defined as $L(P) = \sum_{v_i} w(v_i)$
- Critical Path is the longest path in the task dependency graph $G = (V, E, w)$ with maximal path length
$$\operatorname{argmax}_{P \in \mathcal{P}} L(P)$$
- $ES(v)$ = Early start of a task
- $LS(v)$ = Latest start of a task
- $EF(v)$ = Early finish of a task
- $LF(v)$ = Latest finish of a task

Task Dependency Graph Example



Task Dependency Graph Example

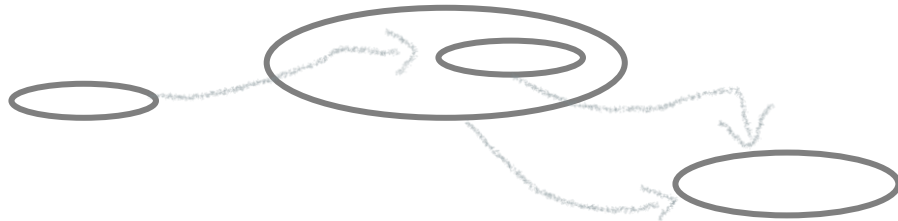


Task/Vertex	Weight	Direct Predecessor	Early Start	Early Finish	Late Start	Late Finish
v_1	3	$\emptyset$	0	3	0	3
v_2	4	$\emptyset$	0	4	2	6
v_3	3	v_1	3	6	3	6
v_4	1	v_2, v_3	6	7	6	7
v_5	2	v_3, v_4	7	9	7	9
v_6	5	v_4, v_5	9	14	9	14

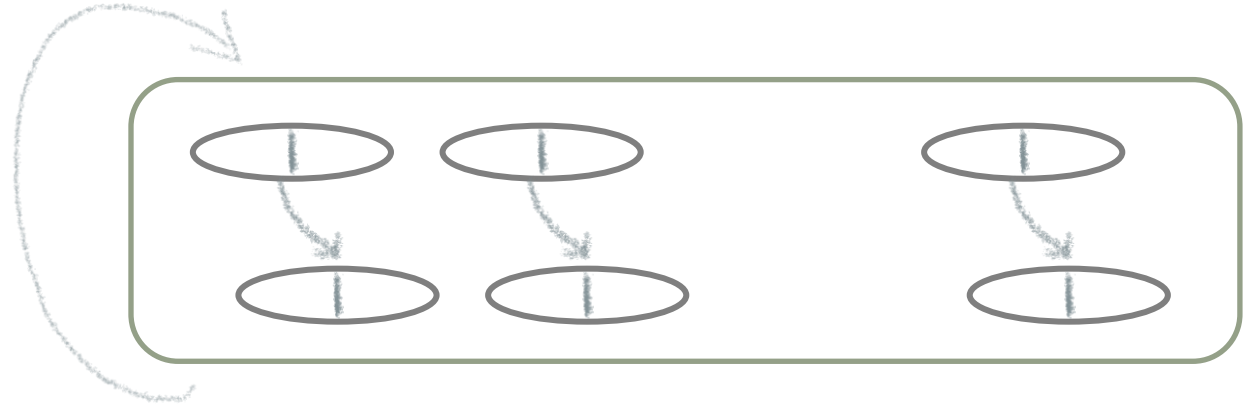
OpenMP 6.0 - Loops, Task Graphs, ...

OpenMP 6.0 provides several new features:

- Loop Fusion/Tiling/Unrolling/....
- User-Defined Inductions.
- Task Graphs to record tasks (+dependencies) and optimize repeated execution of tasks.
- Transparent Tasks inheriting dependencies.



- ...
- For more information, see:
 - <https://www.openmp.org/>
 - www.youtube.com/watch?v=YTbPPhOjO58&t=687s



```
while (residual > Tolerance) {  
    #pragma omp taskgraph graph_id(0)  
    {  
        for (j=0;j<A_SIZE;++j){  
            #pragma omp task depend (out:A[j])  
            preprocess(A[j]);  
            #pragma omp task depend (out:A[j])  
            compute(A[j]);  
        }  
    }  
    residual = calc_residual(A);  
}
```